



ANALYSIS OF DESIGN AND ALGORITHMS

Submitted By: Raman

Course : B.Tech CSE Data Science

Semester : IV

Roll No : 2401420021

Submitted To: Aarti Sangwan

LAB EXPERIMENT 3 – GRAPH ALGORITHM

◇ 1. AIM

To implement and analyze graph algorithms including:

- Graph Representation
 - BFS and DFS Traversals
 - Topological Sorting
 - Shortest Path Algorithms
 - Minimum Spanning Tree Algorithms
-

◇ 2. OBJECTIVES

- To represent graphs using adjacency matrix and list
 - To implement traversal algorithms (BFS & DFS)
 - To solve dependency problems using topological sorting
 - To compute shortest paths using Dijkstra and Bellman-Ford
 - To construct MST using Prim and Kruskal
-

◇ 3. SOFTWARE

REQUIREMENTS

- Python 3.x

- Jupyter Notebook
 - Libraries: time, heapq, collections
-

◇ 4. THEORY

◇ Graph

A graph consists of:

- **Vertices (nodes)**
 - **Edges (connections)**
-

◇ Graph Representation

1. **Adjacency Matrix** → 2D array
 2. **Adjacency List** → Dictionary
-

◇ BFS

- Uses Queue
- Level-wise traversal

◇ DFS

- Uses recursion/stack
 - Depth-first traversal
-

◇ Topological Sorting

Used for ordering tasks in DAG
(Directed Acyclic Graph)

◇ Shortest Path

- Dijkstra → Non-negative weights
- Bellman-Ford → Handles negative weights

◇ Minimum Spanning Tree

- Prim's Algorithm
 - Kruskal's Algorithm
-

◇ 5. IMPLEMENTATION

◇ TASK 1: GRAPH REPRESENTATION

◇ Code

Adjacency Matrix

```
# Adjacency Matrix
graph_matrix = [
    [0, 1, 1],
    [1, 0, 0],
    [1, 0, 0]
]

# Adjacency List
graph_list = {
    'A': ['B', 'C'],
    'B': ['A'],
    'C': ['A']
}

print("Adjacency Matrix:")
for row in graph_matrix:
    print(row)

print("\nAdjacency List:")
for key in graph_list:
    print(key, "->", graph_list[key])
```

Adjacency Matrix:
[0, 1, 1]
[1, 0, 0]
[1, 0, 0]

Adjacency List:
A -> ['B', 'C']
B -> ['A']
C -> ['A']

◇ Observation

Matrix → Uses more space ($O(V^2)$)

List → More efficient ($O(V+E)$)

◇ TASK 2: BFS AND DFS

◇ Code (BFS)

from collections import deque

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            queue.extend(graph[node])

graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['E'],
    'D': [],
    'E': []
}

print("BFS Traversal:")
bfs(graph, 'A')
```

✓ 00%

BFS Traversal:
A B C D E

◇ Code (DFS)

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

print("\nDFS Traversal:")
dfs(graph, 'A')
```

✓ 00%

DFS Traversal:
A B D C E

◇ Observation

BFS explores level-wise

DFS explores depth-wise

Both have time complexity $O(V + E)$

◆ TASK 3: TOPOLOGICAL SORTING

◆ Code

```
def topo_sort(graph):
    visited = set()
    stack = []

    def dfs(node):
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                dfs(neighbor)
            stack.append(node)

    for node in graph:
        dfs(node)

    return stack[::-1]

graph = {
    'A': ['C'],
    'B': ['C'],
    'C': ['D'],
    'D': []
}

print("Topological Sort:", topo_sort(graph))
✓ 00s
Topological Sort: ['B', 'A', 'C', 'D']
```

◆ Observation

- Applicable only to DAG
 - Used in scheduling problems
-

◆ TASK 4: SHORTEST PATH ALGORITHMS

◆ (A) DIJKSTRA ALGORITHM

◆ Code

```

import heapq

def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    pq = [(0, start)]

    while pq:
        curr_dist, node = heapq.heappop(pq)

        for neighbor, weight in graph[node]:
            distance = curr_dist + weight
            if distance < dist[neighbor]:
                dist[neighbor] = distance
                heapq.heappush(pq, (distance, neighbor))

    return dist

graph = {
    'A': [('B',1), ('C',4)],
    'B': [('C',2), ('D',5)],
    'C': [('D',1)],
    'D': []
}

print("Dijkstra Shortest Paths:", dijkstra(graph, 'A'))

```

0.0s

Dijkstra Shortest Paths: {'A': 0, 'B': 1, 'C': 3, 'D': 4}

◇ (B) BELLMAN-FORD ALGORITHM

◇ Code

```

def bellman_ford(edges, V, start):
    dist = [float('inf')] * V
    dist[start] = 0

    for _ in range(V-1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # Negative cycle check
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            print("Negative cycle detected")
            return None

    return dist

edges = [
    (0,1,4),
    (0,2,5),
    (1,2,-3),
    (2,3,4)
]

print("Bellman-Ford Result:", bellman_ford(edges, 4, 0))

```

0.0s

Bellman-Ford Result: [0, 4, 1, 5]

◇ Observation

- Dijkstra is faster but cannot handle negative weights
- Bellman-Ford handles negative weights

◇ TASK 5: MINIMUM SPANNING TREE

◇ (A) PRIM'S ALGORITHM

◇ Code

```
import heapq

def prim(graph, start):
    visited = set([start])
    edges = []
    pq = []

    for to, weight in graph[start]:
        heapq.heappush(pq, (weight, start, to))

    mst = []

    while pq:
        weight, frw, to = heapq.heappop(pq)
        if to not in visited:
            visited.add(to)
            mst.append((frw, to, weight))

            for next_to, w in graph[to]:
                heapq.heappush(pq, (w, to, next_to))

    return mst

graph = {
    'A': [('B', 2), ('C', 3)],
    'B': [('A', 2), ('C', 1), ('D', 4)],
    'C': [('A', 3), ('B', 1), ('D', 5)],
    'D': [('B', 4), ('C', 5)]
}

print("Prim MST:", prim(graph, 'A'))
```

✓ 0.0s

Prim MST: [('A', 'B', 2), ('B', 'C', 1), ('B', 'D', 4)]

◇ (B) KRUSKAL'S ALGORITHM

◇ Code


```

def find(parent, i):
    if parent[i] == i:
        return i
    return find(parent, parent[i])

def union(parent, rank, x, y):
    rootX = find(parent, x)
    rootY = find(parent, y)

    if rank[rootX] < rank[rootY]:
        parent[rootX] = rootY
    else:
        parent[rootY] = rootX
        if rank[rootX] == rank[rootY]:
            rank[rootX] += 1

def kruskal(edges, V):
    edges.sort(key=lambda x: x[2])
    parent = list(range(V))
    rank = [0]*V

    mst = []

    for u, v, w in edges:
        if find(parent, u) != find(parent, v):
            mst.append((u, v, w))
            union(parent, rank, u, v)

    return mst

edges = [
    (0,1,2),
    (0,2,3),
    (1,2,1),
    (1,3,4),
    (2,3,5)
]

print("Kruskal MST:", kruskal(edges, 4))

```

✓ 0.0s

Kruskal MST: [(1, 2, 1), (0, 1, 2), (1, 3, 4)]

◇ Observation

- Prim → Better for dense graphs
 - Kruskal → Better for sparse graphs
-

◇ 6. RESULT

All graph algorithms were successfully implemented and tested.

◇ 7. CONCLUSION

- Graph algorithms are essential in real-world systems

- BFS/DFS are used for traversal and connectivity
- Dijkstra and Bellman-Ford solve shortest path problems
- MST algorithms optimize network costs